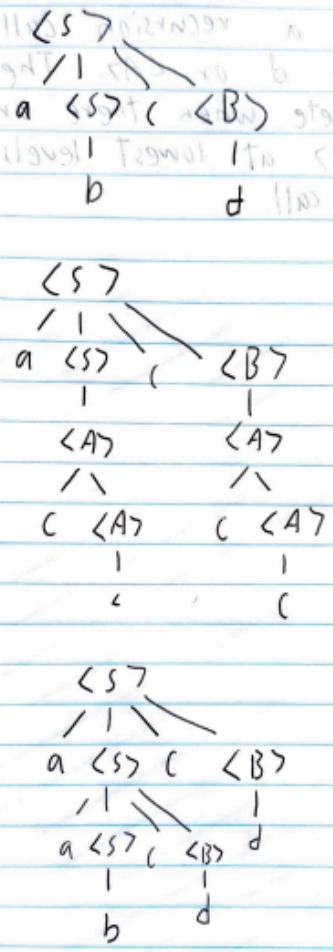
Winston Shih (WXS190012)
Satyam Garg (SXG210250)

CS 4337 HW 2

1. Categories for evaluating a language
   - Readability -
     - Java is a highly readable language because of its explicit mentions of keywords like class, public, and import. Java also tends to be very wordy. For example “public static void main(String[] args)” is too verbose that it makes the code look complex and a lot to handle.
   - Writability -
     - Java has a lot of integration with other libraries which makes it extensive and easy to use or write. The integration of libraries gives the user a great deal of power for tasks to program. Because of the verbose nature of the language (as mentioned in the readability section), it can be slower to develop compared to python for example.
   - Reliability -
     - Java has error checking when compiling so it is quite reliable. Java also manages trash collection which saves the user from memory leaks and any dangling pointer issues.
   - Cost -
     - Due to the verbose nature of java, it has a decent bit of syntax to worry about, so training programmers in Java can be a lot of work. The actual cost to execute Java can be a factor since Java runs on a JVM. Running on a JVM causes the code to be slower than other programming languages, but the cost of maintainability is low due to readability and high reliability.
   - Portability -
     - Since Java runs on a JVM, a Java program is portable from one device to another as long as the java environment is set. Java is also portable across operating systems.
   - Well-definedness -
     - Java is quite an old language so that means it has a lot of documentation. Therefore, Java is well defined and also the language’s documentation helps it behave consistently meaning that Java is also portable as well as reliable.
   - Generality -
     - Java is highly versatile, meaning that it can be used in many settings ranging from backend systems, mobile phone development and even large scale projects. Java has a lot of versatile uses, so it can be generalized in a wide variety of ways.

2.

2.



You start at <S> and call one of its 3 definitions: a <S> c <B>, b, or <A>. If there is an <A>, that is a recursion call to <A>'s definitions which are c <A> or c. If there is a <B>, that is a recursion call to <B>'s definitions: d or <A>. The parse tree is complete when there are no <S>, <A>, or <B> at lowest levels of tree to recursively call.

3.

3. Msw (<expr>, s) Δ= case <expr> of

<dec_num> -> Mdec(<dec_num>, s)

<var> -> if VARMAP (X, s) == undef then error

else VARMAP(<var>, s)

(case <exp> of <cond_exp>

-> if Mst(L, s) == error then error

else Mst (L, s)

<default_exp> -> if Mst (L, s) == error)

then error

<bin_expr> -> if (Mst (X, s) == error)

then error

else case (if (<expr>.

<cond-exp[1] == error))

then error

else (<expr>. <default-exp> == error))

then error

M_1 (switch B do L, S) → state before switch case

should return a new state after executing switch case (iteration).

M_B (B, S) → function evaluates B at state S and returns true or false or error.

M_{sw} (W, S) → the function evaluates W one time and returns a new state.

int Sum = 0, n = 1;

switch (n) {

case 0:

B

L

```
        sum += 1;
    case 1:
        sum += 2;
    default:
        sum += 0;
}
```

state before switch case -> $S_0 = \{\langle sum, 0\rangle, \langle n, 1\rangle\}$

state after switch case -> $S_1 = \{\langle sum, 2\rangle, \langle n, 1\rangle\}$

int sum = 2, n = 2; -> $S = \{\langle sum, 2\rangle \langle n, 2\rangle\}$

```
switch (n) {
    case 0:
        sum += 1;
    case 1:
        sum += 2;
```

no change in state

```
    default:
        sum += 0;
}
```

no change in state

4A.
x=4y+2
y=3x-6
y>6
3x-6>6
3x>12
x>4
wp(y=3x-6, y>6)=x>4
4y+2>4
4y>2
y>1/2
wp(x=4y+2, x>4)=y>1/2 if y>6

4B.
if(x==y) y=2x+4 else y=8x-4 {y>4}
if x==y:
2x+4>4
2x>0
x>0
wp(y=2x+4, y>4)=x>0
Else:
8x-4>4
8x>8
x>1
wp(y=8x-4, y>4)=x>1
(x==y→x>0)^(x=/=y→x>1)
=((x==y)→(x>0))^((x=/=y)→(x>1))
=(x=y^x>0) ∨ (x=/=y^x>1) if y>4
5.
EBNF rules:
<function> -> <type> ID '(' <params> ')' '{' <stmt_list> '}'
<params> -> <type> ID '[' ']' {',' <type> ID}
<type> -> int
<stmt_list> -> {<stmt>}
<stmt> -> <declaration> | <while_loop> | <if_stmt> | <return_stmt> | <expr>';'
<declaration> -> <type> ID '=' <expr>';'
<while_loop> -> while '(' <expr> ')' '{' <stmt_list> '}'
<if_stmt> -> if '(' <expr> ')' '{' <stmt_list> '}'
<return_stmt> -> return <expr>';'
<expr> -> <term> {<rel_op> <term>}
<term> -> <factor> {<add_op> <factor>}
<factor> -> ID ['[' <expr> ']'] | INT_LITERAL | ID'--'
<rel_op> -> '==' | '>=' | '>'
<add_op> -> '+' | '-'

Lexical analysis

Lexeme	Token
int	INT_KEYWORD
duplicate_check	IDENTIFIER
(	LEFT_PAREN
int	INT_KEYWORD
a	IDENTIFIER
[	LEFT_BRACKET
]	RIGHT_BRACKET
,	COMMA
int	INT_KEYWORD
n	IDENTIFIER
)	RIGHT_PAREN
{	LEFT_BRACE
int	INT_KEYWORD
i	IDENTIFIER
=	ASSIGN_OP
n	IDENTIFIER
;	SEMICOLON
while	WHILE_KEYWORD
(	LEFT_PAREN
i	IDENTIFIER
>	REL_OP

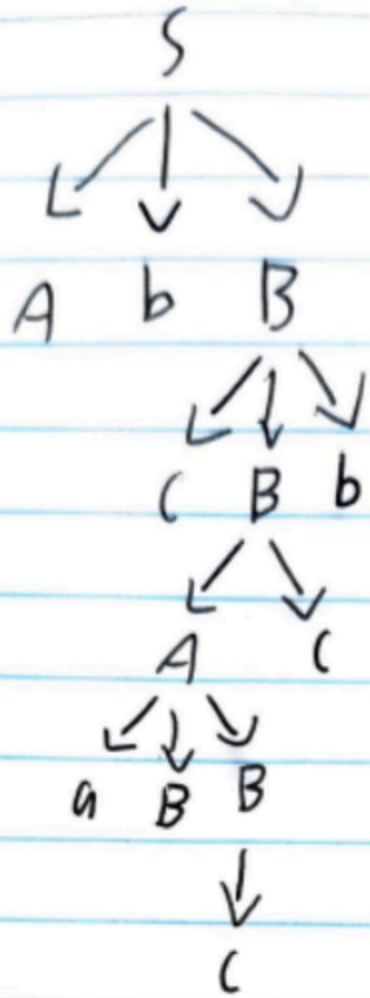
0	INT_LITERAL
)	RIGHT_PAREN
{	LEFT_BRACE
i	IDENTIFIER
--	DECREMENT_OP
;	SEMICOLON
int	INT_KEYWORD
j	IDENTIFIER
=	ASSIGN_OP
i	IDENTIFIER
-	ADD_OP
1	INT_LITERAL
;	SEMICOLON
while	WHILE_KEYWORD
(	LEFT_PAREN
j	IDENTIFIER
>=	REL_OP
0	INT_LITERAL
)	RIGHT_PAREN
{	LEFT_BRACE
if	IF_KEYWORD
(	LEFT_PAREN
a	IDENTIFIER

[	LEFT_BRACKET
i	IDENTIFIER
]	RIGHT_BRACKET
==	REL_OP
a	IDENTIFIER
[	LEFT_BRACKET
j	IDENTIFIER
]	RIGHT_BRACKET
)	RIGHT_PAREN
{	LEFT_BRACE
return	RETURN_KEYWORD
1	INT_LITERAL
;	SEMICOLON
}	RIGHT_BRACE
j	IDENTIFIER
--	DECREMENT_OP
;	SEMICOLON
}	RIGHT_BRACE
}	RIGHT_BRACE
return	RETURN_KEYWORD
0	INT_LITERAL
;	SEMICOLON
}	RIGHT_BRACE

```
PS C:\Users\vibhu>  & 'C:\Program F
n' 'Parser'
token=0 lexeme=int
in func
token=4 lexeme=duplicate_check
token=10 lexeme=(
token=0 lexeme=int
in params
token=4 lexeme=a
token=14 lexeme=[
token=15 lexeme=]
token=16 lexeme=,
token=0 lexeme=int
token=4 lexeme=n
token=11 lexeme=)
out params
token=12 lexeme={
token=0 lexeme=int
in stmtlist
in stmt
in decl
token=4 lexeme=i
token=6 lexeme==
token=4 lexeme=n
in expr
in term
in factor
token=17 lexeme=;
out factor
out term
out expr
token=1 lexeme=while
out decl
out stmt
in stmt
in while
token=10 lexeme=(
token=4 lexeme=i
in expr
Java: Ready
```

6.

6.

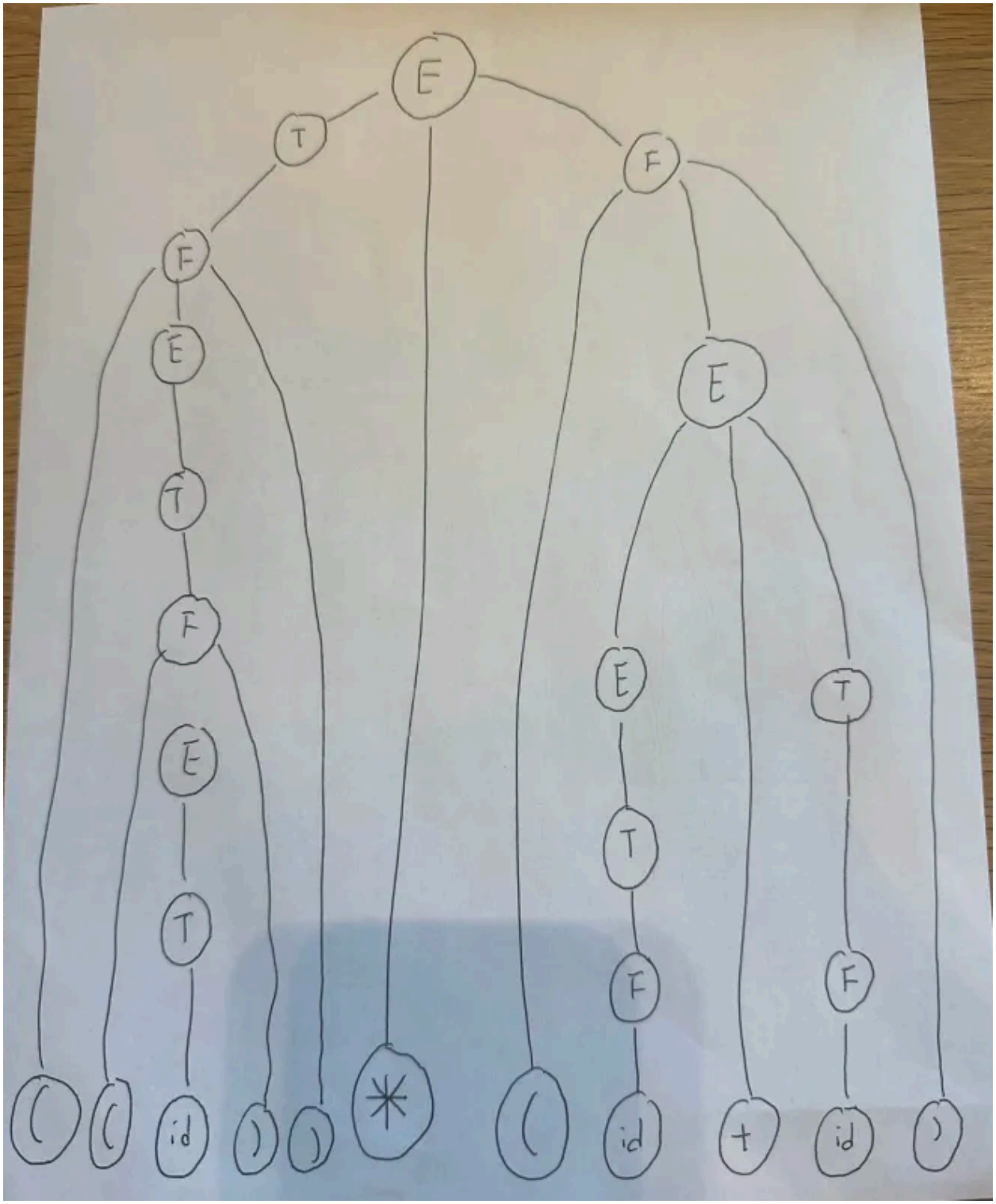


Phrases: AbcBb, cBb, Acb, aBB, c
Simple Phrase: c, aBB, Acb, cBb
Handle: c
7.
((id))*(id+id)

Stack (symbols + states)	**Input**	**Action**
0	((id))*(id+id)$	Shift 4
0(4	(id))*(id+id)$	Shift 4
0(4(4	id))*(id+id)$	Shift 5

0(4(4id5	))*(id+id)$	Reduce 6 (Use GOTO[4,F])
0(4(4F3	))*(id+id)$	Reduce 4 (Use GOTO[4,T])
0(4(4T2	))*(id+id)$	Reduce 2 (Use GOTO[4,E])
0(4(4E8	))*(id+id)$	Shift 11
0(4(4E8)11	)*(id+id)$	Reduce 5 (Use GOTO[4,E])
0(4F3	)*(id+id)$	Reduce 4(Use GOTO[4,T])
0(4T2	)*(id+id)$	Reduce 2 (Use GOTO[4,E])
0(4E8	)*(id+id)$	Shift 11
0(4E8)11	*(id+id)$	Reduce 5 (Use GOTO[4,F])
0F3	*(id+id)$	Reduce 4 (Use GOTO[4,T])
0T2	*(id+id)$	Shift 7
0T2*7	(id+id)$	Shift 4
0T2*7(4	id+id)$	Shift 5
0T2*7(4id5	+id)$	Reduce 6 (Use GOTO[4,F])
0T2*7(4F3	+id)$	Reduce 4 (Use GOTO[4,T])
0T2*7(4T2	+id)$	Reduce 2 (Use GOTO[4,E])

0T2*7(4E8	+id)$	Shift 6
0T2*7(4E8+6	id)$	Shift 5
0T2*7(4E8+6id5	)$	Reduce 6 (Use GOTO[6,F])
0T2*7(4E8+6F3	)$	Reduce 4 (Use GOTO[6,T])
0T2*7(4E8+6T9	)$	Reduce 1 (Use GOTO[4,E])
0T2*7(4E8	)$	Shift 11
0T2*7(4E8)11	$	Reduce 5 (Use GOTO[7,F])
0T2*7F10	$	Reduce 3 (Use GOTO[0,T])
0T2	$	Reduce 2 (Use GOTO[0,E])
0E1	$	Accept


E
T
F
F
E
T
F
E
T
(
(
id
)
)
*
(
E
E
T
T
F
F
id
+
id
)